Easy Travels Frontend Technical Roadmap

End-to-End Implementation, OCR Integration, Hosting & Backend Handoff

Scope: Frontend Phase (UI/UX, OCR, Admin Tracking, Vercel/Netlify Hosting) | Phase 2: Backend (Auth/AFS)

1. Executive Strategy & Technical Scope

This technical roadmap outlines the initial-to-final engineering execution plan for building the frontend of the **Easy Travels** platform. Inspired by global travel visa benchmarks (such as iVisa's 100+ destination workflows, 99% approval tracking, and document wallet features), Easy Travels is architected as an independent, clean-room application.

> **Key Architecture Decision:** Frontend phase is fully decoupled from backend infrastructure. All form steps, OCR card scanning, dynamic styling, and admin tracking mock states operate via client-side state management and WebAssembly. Hosting will be deployed immediately upon frontend completion, followed by Phase 2 backend integration (Authentication, Authorization, and AFS Payment Gateway).

2. Frontend Development Roadmap (Initial to Hosting)

Phase	Core Deliverable & Scope	Tech Stack / Libraries	Milestone Target
Phase 1: Setup & Design Tokens	Configure Next.js 14 / Vite React, Tailwind CSS v4 design tokens, dynamic color themes, borderless card UI, light/dark mode switch.	React 18, Tailwind CSS v4, Lucide Icons, Shadcn UI base.	Week 1
Phase 2: Client Visa Flow	Destination selector (100+ countries), auto-saving step form, passport/visa requirement wizard, document wallet client view.	Zustand state, TanStack Form, LocalStorage cache.	Week 2 - 3
Phase 3: Client OCR Scanning	WebAssembly OCR for Passport Machine Readable Zone (MRZ) & payment card optical recognition directly in browser.	Tesseract.js, mrz-parser, WebCam API canvas overlay.	Week 4
Phase 4: Admin Dashboard	Client application pipeline matrix, status filter (Submitted, Under Review, Approved), document verifier, analytics KPI cards.	TanStack Table v8, Recharts / Chart.js, Mock Service Worker (MSW).	Week 5
Phase 5: Hosting & Deployment	Production build optimization, static site generation (SSG) / edge rendering, custom domain SSL, Vercel/Netlify deployment.	Vercel Edge Platform / Netlify, GitHub Actions CI/CD.	Week 6 (Milestone Completion)

3. Detailed Frontend Module Specifications

3.1 Dynamic & Borderless Styling System

To achieve a premium, aesthetic, and borderless design system, Easy Travels leverages Tailwind CSS v4 CSS variables. Components utilize subtle soft drop-shadows (shadow-sm hover:shadow-md), glassmorphism background blurs (backdrop-blur-md bg-white/80), and seamless container transitions instead of harsh solid borders.

3.2 WebAssembly Client-Side OCR Scan Engine

Easy Travels Frontend Technical Roadmap

End-to-End Implementation, OCR Integration, Hosting & Backend Handoff

Scope: Frontend Phase (UI/UX, OCR, Admin Tracking, Vercel/Netlify Hosting) | Phase 2: Backend (Auth/AFS)

The OCR engine operates entirely within the user's web browser to maximize privacy and responsiveness prior to backend integration.

• **Passport Scanning:** Captures camera feed via navigator.mediaDevices.getUserMedia(), crops the MRZ box, and parses Type 1 & Type 3 passport lines to auto-fill Name, Passport Number, DOB, and Expiry Date.
• **Card Scanning:** Detects credit/visa card boundary boxes, extracts card numbers and expiration dates with image binarization, populating checkout form fields dynamically.

3.3 Admin Control Panel & Client Tracking System

The Admin Dashboard provides real-time oversight of client submissions with comprehensive tracking controls:
• **Application Matrix:** Searchable table featuring client ID, destination country, visa category, submission date, and real-time status pill badges.
• **Document Verifier Drawer:** Side panel overlay for admins to inspect scanned client passports, review OCR accuracy, and change status.
• **System KPIs:** Summary cards tracking total applications, approval rates, average processing times, and daily traffic.

4. Frontend Hosting & CI/CD Deployment Plan

Prior to backend development, the frontend will be deployed on a high-availability serverless global edge platform:

- **Environment Configuration:** Set up .env.production with fallback mock API endpoints and feature flags.
- **Build Optimization:** Execute code-splitting, WebP asset compression, and bundle size reduction (< 150KB initial JS chunk).
- **Edge CDN Deployment:** Deploy to **Vercel / Netlify** with automated GitHub Actions triggered on main branch pushes.
- **SSL & Domain Binding:** Configure custom domain DNS records, automated HTTP/2 SSL certificates, and custom headers.

5. Subsequent Phase: Backend Handoff Blueprint

Once the frontend is hosted and validated, the project transitions smoothly into backend engineering. The frontend code is architected with adapter interfaces ready to connect to live services:

Backend Subsystem	Planned Architecture	Frontend Integration Point
Authentication	OAuth2 + JWT stateless sessions (Node.js / Go microservice).	Replace Mock Auth Provider with AuthContext & HTTP Bearer Interceptor.
Authorization & RBAC	Role-Based Access Control (Client, Agent, Admin permissions).	Enforce Route Guards (<ProtectedRoute roles={['admin']}>).
AFS Payment Gateway	Arab Financial Services (AFS) Merchant API integration.	Connect checkout step to AFS Hosted Payment Page / Tokenized SDK.